

# Practically Groovy: Groovy: A DSL for Java programmers

Write less code and accomplish more with Groovy

Skill Level: Introductory

[Scott Davis](#) ([scott@thirstyhead.com](mailto:scott@thirstyhead.com))

Founder

ThirstyHead.com

17 Feb 2009

Groovy expert Scott Davis reboots the *Practically Groovy* series, dormant since 2006. This initial installment catches you up on Groovy's recent history and the current state of the Groovy union. Then you'll learn how easy it is to get started with Groovy, circa 2009.

Andrew Glover started writing about Groovy for developerWorks in 2004, starting with his introductory "[Feeling Groovy](#)" article in the *alt.lang.jre* series, and continuing with the long-running *Practically Groovy* series. That was before there was a single book on the market about Groovy (there are now more than a dozen), and years before the Groovy 1.0 release in January 2007. Much has changed since the last *Practically Groovy* installment was published in late 2006.

Groovy now averages roughly 35,000 downloads a month. Conservative companies like Mutual of Omaha have more than 70,000 lines of Groovy code in production. Groovy has one of the busiest mailing lists at Codehaus.org, where the project is hosted (see [Resources](#)). The only project that has more downloads and a busier mailing list is Grails, a popular Web framework implemented in Groovy (see [Resources](#)).

Running non-Java™ languages on the JVM is not only commonplace, it's a core part of Sun's strategy for the JVM. Groovy joins Sun-supported languages like JavaScript, JavaFX, JRuby, and Jython in the alternative-languages cavalcade. What was experimental in 2004 is now state-of-the-art.

Writing about Groovy in 2009 is, in many ways, much the same as it was when Andy started. The syntax stabilized in 2005 and remains the same today. New, compelling features are added with each release, but preserving backward compatibility is paramount to the project leaders. This solid foundation makes Groovy an easy choice for Java development shops that have come to rely on technologies that will be around for as long as their applications are in production.

This article aims to get experienced Java developers up to speed quickly as Groovy developers. Don't be fooled by its primer-ish aspect. This series, as its name implies, is all about practical uses for Groovy know-how. After you say "Hello, World" in this gentle beginning, prepare to enter the real world posthaste.

### About this series

Groovy is a modern programming language that runs on the Java platform. It offers seamless integration with existing Java code while introducing dramatic new features like closures and metaprogramming. Put simply, Groovy is what the Java language would look like had it been written in the 21st century.

The key to incorporating any new tool into your development toolkit is knowing when to use it and when to leave it in the box. Groovy can be extremely powerful, but only when applied properly to appropriate scenarios. To that end, the [Practically Groovy](#) series explores the practical uses of Groovy, helping you learn when and how to apply them successfully.

## Installing Groovy

If you've never worked with Groovy before, the first thing you need to do is get it installed. The installation steps are pretty straightforward. They're the same steps you use to install common Java applications like Ant and Tomcat, and even the Java platform itself:

1. [Download](#) the latest Groovy ZIP file or tarball.
2. Unzip the archive to the directory of your choice. (You should avoid directories with spaces in the name.)
3. Create a GROOVY\_HOME environment variable.
4. Add GROOVY\_HOME/bin to the PATH.

Groovy runs best on Java 5 or 6. Type `java -version` at a command prompt to confirm that you are up-to-date. Then type `groovy -version` to make sure that Groovy is properly installed.

All of the major IDEs (Eclipse, IntelliJ, and NetBeans) have a Groovy plug-in that supports features like autocomplete and step-debugging. Although a good IDE is almost a requirement for writing Java code these days, the same isn't necessarily true when it comes to Groovy. Thanks to the conciseness of the Groovy language, many people choose to use a simple text editor instead. Popular open source editors like vi and Emacs offer Groovy support, as do inexpensive commercial text editors like Textpad (for Windows®) and TextMate (for Mac OS X). (See [Resources](#) for more information.)

As you'll see later in the article, incorporating Groovy with an existing Java project is easy. All you need to do is add a single Groovy JAR from GROOVY\_HOME/embeddable to the classpath and wrap the existing javac Ant task in a groovyc task. (Maven offers similar support.)

But before you get too far ahead of yourself, I'll start you off with the obligatory "Hello World" example.

## Hello Groovy World

You know what a "Hello World" example is supposed to demonstrate — it's the simplest possible program that you can write in a given language. What's interesting about "Hello World" in Java code, shown in Listing 1, is how much intermediate language knowledge you need in order to understand fully what's going on:

### Listing 1. A "Hello World" example written in Java code

```
public class HelloJavaWorld{
    public static void main(String[] args){
        System.out.println("Hello Java World");
    }
}
```

You start by creating a file named HelloJavaWorld.java and typing `public class HelloJavaWorld`. The first hard lesson many beginning Java developers learn is that if the class name and file name don't match exactly (including upper- and lowercase), the class won't compile. Also, curious students begin asking about access modifiers like `public` and `private` at this point.

The next line — `public static void main(String[] args)` — generally unleashes an avalanche of questions about implementation details: What is `static`? What is `void`? Why does the method need to be named `main`? What is a `String` array? And finally, try explaining to a first-time Java developer that `out` is a `public, static, final` instance of a `PrintStream` object on the `System` class. I'll never forget the student who said, "Sheesh! All I wanted to do was say 'Hello.'"

Contrast this with "Hello World" in Groovy. Create a file named

HelloGroovyWorld.groovy and type the line shown in Listing 2:

### Listing 2. A "Hello World" example written in Groovy

```
println "Hello Groovy World"
```

Yes, this is the equivalent in Groovy to the Java example in Listing 1. In this case, all of the implementation details — the "baggage" that doesn't immediately contribute to solving the problem at hand — fade into the background, leaving you with code that simply says, "Hello." Type `groovy HelloGroovyWorld` to confirm that it works.

This trivial example demonstrates the twin value propositions of Groovy: it dramatically reduces the lines of code you need to write while it preserves the semantics of the Java equivalent. In the next section, you'll explore this idea further.

## Digging deeper into Hello World

Experienced Java developers know that you must compile your code before it will run on the JVM. Yet, there doesn't appear to be a class file for the Groovy script anywhere. Does that mean that you can execute Groovy source code directly? The answer is, "Not really, but it sure looks that way, doesn't it?"

The Groovy interpreter compiles the source code in-memory before handing it off to the JVM. You can manually perform this step by typing `groovyc HelloGroovyWorld.groovy`. However, if you try to run the resulting class using `java`, you'll be greeted with the exception shown in Listing 3:

### Listing 3. Trying to run a compiled Groovy class without the Groovy JAR on the CLASSPATH

```
$ java HelloGroovyWorld
Exception in thread "main" java.lang.NoClassDefFoundError: groovy/lang/Script
```

As I mentioned earlier, the Groovy JAR must be included in the CLASSPATH. Try it again, this time passing in the `-classpath` argument to `java`, as shown in Listing 4:

### Listing 4. Successfully running a compiled Groovy class with the java command

```
//For UNIX, Linux, and Mac OS X
$ java -classpath $GROOVY_HOME/embeddable/groovy-all-x.y.z.jar:. HelloGroovyWorld
Hello Groovy World

//For Windows
```

```
$ java -classpath %GROOVY_HOME%/embeddable/groovy-all-x.y.z.jar;. HelloGroovyWorld
Hello Groovy World
```

Now you're getting somewhere. But to prove that the Groovy script really is preserving the semantics of the Java example, you need to dig deeper into the bytecode. To start, type `javap HelloJavaWorld`, as shown in Listing 5:

#### Listing 5. Examining the Java bytecode

```
$ javap HelloJavaWorld
Compiled from "HelloJavaWorld.java"
public class HelloJavaWorld extends java.lang.Object{
    public HelloJavaWorld();
    public static void main(java.lang.String[]);
}
```

There shouldn't be too many surprises here, other than some of the niceties that the `javac` compiler added on your behalf. You didn't need to type `extends java.lang.Object` explicitly or provide a default constructor for the class.

Now, type `javap HelloGroovyWorld`, as shown in Listing 6:

#### Listing 6. Examining the Groovy bytecode

```
$ javap HelloGroovyWorld
Compiled from "HelloGroovyWorld.groovy"
public class HelloGroovyWorld extends groovy.lang.Script{
    ...
    public static void main(java.lang.String[]);
    ...
}
```

#### What is a DSL?

Martin Fowler popularized the idea of domain-specific languages (see [Resources](#)). He defines a DSL as a "computer programming language of limited expressiveness focused on a particular domain." By "limited expressiveness," he doesn't mean limited usefulness, but rather that the language only has enough vocabulary to make it appropriate for a "particular domain." A DSL is a tiny specific-purpose language, in contrast to a large general-purpose language like the Java language.

SQL is a great example of a DSL. You couldn't write an operating system in SQL, but it's ideal for the limited domain of dealing with relational databases. In much the same sense, Groovy is a DSL for the Java platform in that it is ideal for the limited domain of Java development. My use of *DSL* here is meant to be more evocative than exact. Perhaps if I call Groovy *an internal DSL for common Java idioms*, I'll lessen the inevitable wrath of DSL purists.

Dave Thomas further clarifies the notion of DSLs (see [Resources](#)). He writes, "Whenever domain experts communicate ... they are speaking jargon, a specialized language that they've invented as a

shorthand for communicating effectively with their peers." Perhaps saying that Groovy is "shorthand Java" comes closer to explaining the relationship between Groovy and the Java language. This article's next section gives another example of this.

Here, you can see that the `groovyc` compiler took the name of your source file and created a class of the same name. (The fact that the class extends `groovy.lang.Script` instead of `java.lang.Object` should help you understand why trying to run the file without the Groovy JAR on the `CLASSPATH` threw the `NoClassDefFoundError` exception.) Amidst all of the other compiler-provided methods, you should be able to find a good old `public static void main(String[] args)` method. The `groovyc` compiler wrapped the lines of your script in this method to preserve Java semantics. This means that you can take advantage of all of your existing Java knowledge when it comes to Groovy.

For example, here's how you can accept command-line input in a Groovy script. Create a new file named `Hello.groovy` and add the line in Listing 7:

#### Listing 7. A Groovy script that accepts command-line input

```
println "Hello, " +  
args[0]
```

Now type `groovy Hello Jane` from the command line. The `args` `String` array is there, as any Java developer would expect. Using `args` here might not make sense to the uninitiated, but it makes perfect sense to seasoned Java developers.

Groovy boils Java code down to its bare essence. The Groovy script that you just wrote is almost like executable pseudocode. It is simple enough on the surface for novices to understand, but it doesn't strip away the underlying power of the Java language for experienced developers. This is what leads me to call Groovy a domain-specific language (DSL) for the Java platform. (See the ["What is a DSL?"](#) sidebar.)

## Plain Old Groovy Objects

JavaBeans — or more casually Plain Old Java Objects (POJOs) — are a mainstay of Java development. You should follow a well-defined set of expectations when you create a POJO to represent a domain object. The class should be `public`, and the fields should be `private` with a corresponding set of `public` getter and setter methods. Listing 8 shows a typical Java POJO:

#### Listing 8. A Java POJO

```
public class JavaPerson{
    private String firstName;
    private String lastName;

    public String getFirstName(){ return firstName; }
    public void setFirstName(String firstName){ this.firstName = firstName; }

    public String getLastName(){ return lastName; }
    public void setLastName(String lastName){ this.lastName = lastName; }
}
```

Plain Old Groovy Objects (POGOs) are a drop-in replacement for POJOs. They perfectly retain the semantics of the POJO while drastically reducing the amount of code you need to write. Listing 9 shows a "shorthand" person class written in Groovy:

### Listing 9. A Groovy POGO

```
class GroovyPerson{
    String firstName
    String lastName
}
```

All classes in Groovy are `public` unless you specify otherwise. All properties are `private`, and all methods are `public`. The compiler provides a set of `public` getter and setter methods for each property automatically. Compile `JavaPerson` with `javac` and `GroovyPerson` with `groovyc`. Now run them both through `javap` to confirm that the Groovy example has everything that the Java example does, right down to extending `java.lang.Object`. (You didn't specify a class in the earlier `HelloGroovyWorld` example, so Groovy created a class that extended `groovy.lang.Script` instead.)

All of this means that you can immediately begin using POGOs as a replacement for your POJOs. The Groovy class is the Java class boiled down to its bare essence. Once the Groovy class is compiled, other Java classes can use it as easily as if it had been written in Java code. To prove this, create a file named `JavaTest.java` and add the code in Listing 10:

### Listing 10. Calling Groovy classes from Java code

```
public class JavaTest{
    public static void main(String[] args){
        JavaPerson jp = new JavaPerson();
        jp.setFirstName("John");
        jp.setLastName("Doe");
        System.out.println("Hello " + jp.getFirstName());

        GroovyPerson gp = new GroovyPerson();
        gp.setFirstName("Jane");
        gp.setLastName("Smith");
        System.out.println("Hello " + gp.getFirstName());
    }
}
```

Even though the getters and setters don't appear in the Groovy source code, this test proves that they are there and fully functional in the compiled Groovy class. But this example wouldn't be complete if I didn't show you the corresponding test in Groovy. Create a file named `TestGroovy.groovy` and add the code in Listing 11:

### Listing 11. Calling Java classes from Groovy

```
JavaPerson jp = new JavaPerson(firstName:"John", lastName:"Doe")
println "Greetings, " + jp.getFirstName() + ".
    It is a pleasure to make your acquaintance."

GroovyPerson gp = new GroovyPerson(lastName:"Smith", firstName:"Jane")
println "Howdy, ${gp.firstName}. How the heck are you?"
```

The first thing you probably notice is the new constructor that Groovy offers, allowing you to name the fields and specify them in any order you'd like. Even more interesting is the fact that you can use this constructor on either Java or Groovy classes. How is this possible? In reality, Groovy calls the default no-argument constructor first and then calls the appropriate setter for each field. You could approximate similar behavior in the Java language, but because the Java language lacks named arguments and both fields are `Strings`, you can't pass in the first- and last-name fields in any order.

Next, notice that Groovy supports the traditional, Java way to do `String` concatenation, as well as the Groovy way by embedding code surrounded by `${}` directly in the `String`. (These are called `GStrings`, short for *Groovy Strings*.)

Finally, you see one more bit of Groovy syntactic sugar you can use when calling getters on a class. Instead of using the more verbose `gp.getFirstName()`, you can simply call `gp.firstName`. It may look like you are directly accessing the field, but in reality you are calling the corresponding getter method behind the scenes. Setters work the same way: `gp.setLastName("Jones")` and `gp.lastName = "Jones"` are equivalent, with the latter calling the former under the covers.

I expect that you'll agree that in each case, Groovy feels like a shorthand version of the Java language — one that "domain experts" might use to "communicate effectively with their peers," or something akin to casual banter among old friends.

## Groovy is Java code at the end of the day

One of the most underappreciated aspects of Groovy is the fact that it fully supports Java syntax. As I mentioned earlier, you don't need to unlearn a bit of your Java knowledge when working with Groovy. When you are getting started with Groovy, much of your code will end up looking just like traditional Java code. But as you get

more comfortable with the newer syntax, your code will gradually evolve to embrace the more concise, expressive Groovy style.

To prove that your Groovy code can look exactly like your Java code, copy `JavaTest.java` to `JavaTestInGroovy.groovy`, and type `groovy JavaTestInGroovy`. You should see identical output, but notice that you didn't need to compile the Groovy class before running it.

This demonstration should make Groovy an almost no-brainer choice for experienced Java developers. Because Java syntax is valid Groovy syntax as well, the initial learning curve is practically nonexistent. You can use your existing Java version with Groovy, your existing IDE, and your existing production environment. This means that the disruption to your daily routine is minimal. All you need to do is ensure that the Groovy JAR is somewhere on your CLASSPATH and tweak your build script so that the Groovy classes are compiled along with the Java ones. The next section shows you how to add the `groovyc` task to your Ant build.xml file.

## Compiling Groovy code with Ant

If `javac` were a pluggable compiler, you could instruct it to compile both your Groovy and Java files at the same time. Because it's not, you can simply wrap the `javac` task in Ant with a `groovyc` task. This allows `groovyc` to compile the Groovy source code, and `javac` to compile the Java source code as it always did.

Of course, `groovyc` can compile both Java and Groovy files, but remember the additional convenience methods that `groovyc` added to `HelloGroovyWorld` and `GroovyPerson`? These additional methods would also be added to the Java classes. It's probably best to let `groovyc` compile the Groovy files and let `javac` compile the Java ones.

To call `groovyc` from Ant, define the task using `taskdef` and then use the `groovyc` task as you would normally use the `javac` task (see [Resources](#) for more information). Listing 12 shows the Ant build script:

### Listing 12. Compiling Groovy and Java code with Ant

```
<taskdef name="groovyc"
         classname="org.codehaus.groovy.ant.Groovyc"
         classpathref="my.classpath"/>

<groovyc srcdir="${testSourceDirectory}" destdir="${testClassesDirectory}">
  <classpath>
    <pathelement path="${mainClassesDirectory}"/>
    <pathelement path="${testClassesDirectory}"/>
    <path refid="testPath"/>
  </classpath>
  <javac debug="on" />
</groovyc>
```

By the way, those `Strings` with `${}` inside them look suspiciously like `GStrings`, don't they? Groovy is a best-of-breed language, shamelessly borrowing syntax and features from a variety of other languages and libraries. This isn't the last time you'll see something in Groovy that makes you say, "Hmm, haven't I seen that somewhere before?"

## Conclusion

This has been a whirlwind tour of Groovy. You learned a bit about where Groovy has been and where it stands today. You got Groovy installed on your system, and through a couple of simple examples you caught just a glimpse of the power that Groovy offers Java developers.

Groovy isn't the only alternate language that runs on the JVM. JRuby is a great solution for Java developers who already know Ruby. Jython is a great solution for Java developers who already know Python. But as you saw, Groovy is a great solution for Java developers who already know the Java language. The fact that Groovy offers a concise Java-like syntax that also preserves Java semantics is pretty compelling. And, a new language whose path to adoption doesn't involve `del *`, `* or rm -Rf *` is a nice change, wouldn't you say?

Next time, you'll learn about iteration in Groovy. Your code often needs to walk through things item by item, whether it's a list, a file, or an XML document. You'll see the pervasive `each` closure up close. Until then, I hope that you find plenty of practical uses for Groovy.

# Resources

## Learn

- [Groovy](#): Learn more about Groovy at the project Web site.
- [AboutGroovy.com](#): Keep up with the latest Groovy news and article links.
- [Groovy Recipes](#) (Scott Davis, Pragmatic Programmers, 2008): Learn more about Groovy and Grails in Scott Davis' latest book.
- [Mastering Grails](#): Scott Davis's companion series focuses on this Groovy-based platform for Web development.
- [Groovy stats](#): Check out download statistics for Groovy and Grails.
- [Groovy mailing list](#): Browse, search, or subscribe to the Groovy mailing list.
- [Groovy IDE Support](#): Read about IDEs and text editors that support Groovy.
- [Using Domain Specific Languages](#): Find out what Martin Fowler has to say about DSLs.
- "The 'Language' in Domain-Specific Language Doesn't Mean English (or French, or Japanese, or ...)" (Dave Thomas, PragDave, March 2008): DSL fan Thomas weighs in.
- [The groovyc Ant Task](#): Learn more about using Ant to compile Groovy code.
- [Technology bookstore](#): Browse for books on these and other technical topics.
- [developerWorks Java technology zone](#): Find hundreds of articles about every aspect of Java programming.

## Get products and technologies

- [Groovy](#): Download the latest Groovy ZIP file or tarball.

## Discuss

- Check out [developerWorks blogs](#) and get involved in the [developerWorks community](#).

# About the author

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of [ThirstyHead.com](#), a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At](#)

[Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#) and [Practically Groovy](#).

## Trademarks

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.